

# Техническое задание на разработку классов `BigInteger` и `Rational`

Продвинутый курс C++, ФИВТ/ФПМИ МФТИ

5 июля 2026 г.

## 1 Введение

В рамках данного задания необходимо реализовать два класса — `BigInteger` (длинная арифметика для целых чисел) и `Rational` (дроби произвольной точности), а также сопутствующие операторы. Реализация должна быть эффективной, не содержать утечек памяти и не приводить к неопределенному поведению (UB).

---

## 2 Часть 1. Класс `BigInteger`

Класс должен поддерживать работу с целыми числами произвольной длины и знака.

### 2.1 Конструкторы

- `BigInteger()` — инициализация нулем.
- `BigInteger(int)` — инициализация из стандартного целочисленного типа.
- `BigInteger(long long)` — инициализация из типа `long long`.
- `BigInteger(const std::string&)` — инициализация из строки (может содержать знак `-`).

### 2.2 Преобразования типов

- Явное приведение к `bool` (`explicit operator bool() const`) — возвращает `false`, если число равно нулю, иначе `true`.
- Метод `std::string toString() const` — возвращает строковое представление числа.

### 2.3 Операции сравнения

Должны быть реализованы все 6 операторов сравнения. Они не должны изменять объекты (`const`):

- `bool operator==(const BigInteger&, const BigInteger&)`
- `bool operator!=(const BigInteger&, const BigInteger&)`

- `bool operator<(const BigInteger&, const BigInteger&)`
- `bool operator>(const BigInteger&, const BigInteger&)`
- `bool operator<=(const BigInteger&, const BigInteger&)`
- `bool operator>=(const BigInteger&, const BigInteger&)`

## 2.4 Арифметические операторы

Рекомендуется сначала объявить операторы с присваиванием (`+=`), а бинарные (`+`) реализовать через них.

- Унарный минус: `BigInteger operator-() const`
- Префиксный и постфиксный инкремент/декремент: `++` и `--` (в обеих формах).
- Сложение и вычитание: `+=`, `+`, `-=`, `-`.
- Умножение: `*=`, `*`.
- Деление и остаток от деления: `/=`, `/`, `%=`, `%`.

**Важно:** Деление и остаток должны работать по правилам C++ (деление округляется к нулю, знак остатка совпадает со знаком делимого).

## 2.5 Ввод и вывод

Перегрузите операторы потока:

- `std::ostream& operator<<(std::ostream& out, const BigInteger& num)`
- `std::istream& operator>>(std::istream& in, BigInteger& num)`

—

## 3 Часть 2. Класс Rational

Класс `Rational` описывает обыкновенные дроби произвольной точности. Дробь должна всегда храниться в несократимом виде с положительным знаменателем.

### 3.1 Конструкторы и внутреннее устройство

- Должен хранить два поля типа `BigInteger`: числитель и знаменатель.
- `Rational()` — инициализация нулем ( $0/1$ ).
- `Rational(const BigInteger&)` — из целого числа ( $N/1$ ).
- `Rational(int)` — из `int` через неявное приведение к `BigInteger`.
- Конструктор из двух `BigInteger` (числитель и знаменатель) с автоматическим сокращением на наибольший общий делитель (НОД).

## 3.2 Методы

- `std::string toString() const` — возвращает дробь в виде `числитель/знаменатель` (если знаменатель равен 1, то только числитель).
- `std::string asDecimal(size_t precision) const` — `precision(/truncation)`.
- Явное приведение к `double`: `explicit operator double() const`.

## 3.3 Операторы

- Все арифметические операторы: `+=`, `+`, `-=`, `-`, `*=`, `*`, `/=`, `/`.
  - Унарный минус `-`.
  - Все 6 операторов сравнения: `==`, `!=`, `<`, `>`, `<=`, `>=`.
- 

## 4 Ключевые требования к качеству кода

1. **Эффективность хранения:** Числа должны храниться по крупному основанию (например, в системе счисления по основанию  $10^9$  или  $10^4$ ). Поцифровое хранение в векторе гарантированно приведет к *Time Limit Exceeded (TLE)* на тестах деления и умножения.
2. **Лидирующие нули:** После любых операций контейнер цифр должен очищаться от незначащих нулей. Число 0 должно иметь ровно одну цифру 0 в векторе (или пустой вектор).
3. **Константная корректность:** Все методы, не изменяющие состояние объекта, обязаны быть `const`. Параметры должны передаваться по константной ссылке (`const BigInteger&`).
4. **Сложность умножения:** Стандартное умножение «в столбик» имеет сложность  $\mathcal{O}(N^2)$ . Если тесты будут падать по TLE, может потребоваться оптимизация Карацубы  $\mathcal{O}(N^{\log_2 3})$ .